

Assignment 11.2

HT.NO : 2303A51056

B.NO : 16

Task Description -1 – (Stack Using AI Guidance)

- **Task:** With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Expected Output:

- A Python Stack class supporting push, pop, peek, and empty-check operations with proper documentation.

The screenshot shows a VS Code editor with a Python file named `task1.py` containing a `Stack` class implementation. The code includes docstrings, comments, and example usage. The terminal output shows the execution of the code, demonstrating the push, pop, peek, and empty-check operations. The AI Chat window on the right provides guidance and requirements for the implementation.

```
1 """Simple Stack implementation in Python.
2 This module defines a Stack class with the following operations:
3 - push(item)
4 - pop()
5 - peek()
6 - is_empty()
7 It also includes example usage at the bottom.
8 """
9 class Stack:
10     """A simple LIFO (Last-In, First-Out) stack data structure.
11     Internally this uses a Python list to store elements.
12     """
13     def __init__(self):
14         """Initialize an empty stack.
15
16         Time complexity: O(1)
17         """
18         self.items = []
19     def push(self, item):
20         """Add an item to the top of the stack.
21
22         Args:
23             item: The value to be pushed onto the stack.
24
25         Time complexity: O(1) on average (amortized), because list append is O(1).
26         """
27         self.items.append(item)
28     def pop(self):
29         """Remove the top element from the stack.
30
31         Time complexity: O(1)
32         """
33         return self.items.pop()
34     def peek(self):
35         """Return the top element of the stack without removing it.
36
37         Time complexity: O(1)
38         """
39         return self.items[-1] if self.items else None
40     def is_empty(self):
41         """Check if the stack is empty.
42
43         Time complexity: O(1)
44         """
45         return len(self.items) == 0
46
47 # Example usage
48 stack = Stack()
49 stack.push(10)
50 stack.push(20)
51 stack.push(30)
52 print(f"Current stack: {stack.items}")
53
54 # Popping elements
55 stack.pop()
56 stack.pop()
57 print(f"Current stack: {stack.items}")
58
59 # Peek operation
60 print(f"Top element (peek): {stack.peek()}")
61
62 # Empty check
63 print(f"Is stack empty? {stack.is_empty()}")
64 print(f"Stack size: {len(stack.items)}")
```

Terminal Output:

```
Pushing elements: 10, 20, 30
Current stack: Stack([10, 20, 30])

Top element (peek): 30

Popping elements:
Popped: 30
Popped: 20
Current stack: Stack([10])

Is stack empty? False
Stack size: 1
```

AI Chat Window:

DESIGNING A STACK DATA STRUCTURE IN PYTHON

Tip: Try **Plan mode** to let the agent perform deep analysis and planning before implementing changes.

Act as a Python programming expert.

Help me design and implement a Stack data structure in Python.

Requirements:

- Create a Stack class
- Implement `push()`, `pop()`, `peek()`, and `is_empty()` methods
- Include proper comments explaining each method
- Add example usage with sample output
- Also explain time complexity of each operation

lab1.py

✓ Optimizing tool selection...

✓ Designing a Python Stack.

I'll open your `lab1.py` to either extend or replace it with a clean Stack implementation and example usage.

✓ Reviewed lab1.py

Now I'll write a Stack class with all required methods, comments, and an example usage into `lab1.py`.

✓ Updated lab1.py

Here's what I've added and how it meets your requirements:

task5.py + task1.py

Describe what to build next

GPT-5.1

Implemented a Stack class backed by a Python list.

Methods:

- `push(item)`: adds an element to the top.
- `pop()`: removes and returns the top element, raises `IndexError` if empty.
- `peek()`: returns (without removing) the top element, raises `IndexError` if empty.
- `is_empty()`: returns `True` if there are no elements.

Task Description -2 – (Queue Design)

• **Task:** Use AI assistance to create a Queue data structure

following FIFO principles

Expected Output:

• A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

The screenshot shows a VS Code editor with a Python file named `task2.py` open. The code implements a `Queue` class using `collections.deque` for efficient operations. The chat window on the right provides instructions and feedback on the implementation.

```
1 """Queue implementation in Python following FIFO (First-In, First-Out) principle.
2
3 This module defines a Queue class with the following operations:
4 - enqueue(item)
5 - dequeue()
6 - front()
7 - size()
8
9 It also includes example usage at the bottom.
10 """
11
12 from collections import deque
13
14
15 class Queue:
16     """A simple FIFO (First-In, First-Out) queue data structure.
17
18     Internally this uses collections.deque for efficient O(1) operations
19     at both ends.
20     """
21
22     def __init__(self):
23         """Initialize an empty queue.
24         """
```

CHAT

DESIGNING A STACK DATA STRUCTURE IN PYTHON

Restore Checkpoint

Act as a data structures instructor. Create a Queue data structure in Python following FIFO principles. Requirements:

- Create a Queue class, Implement enqueue(), dequeue(), front(), and size() methods. Add proper comments, Provide example usage with output, Explain how FIFO works in this implementation. Mention time complexity

Click to Edit task2.py

✓ Implementing queue in task2.py

I'll open your task2.py and then implement a commented Queue class with example usage and time complexity notes.

✓ Reviewed task2.py

Now I'll write the Queue implementation, comments, explanation, and example usage into task2.py.

✓ Updated task2.py

Here's your Queue implementation and how it satisfies each requirement:

- Implemented a `Queue` class in `task2.py` using `collections.deque` for efficient operations.
- Methods:
 - `enqueue(item)`: adds an element to the rear of the queue (FIFO).

task2.py task5.py

Describe what to build next

GPT-5.1

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Code + - - - |

C:\Users\akhil\OneDrive\Documents\Devops>python -u "c:\Users\akhil\OneDrive\Documents\Devops\AI-ASSISTED_CODING\task2.py"

Is queue empty? True

Enqueuing elements: 'A', 'B', 'C'

Current queue: Queue(['A', 'B', 'C'])

Front element: A

Dequeuing elements (FIFO order):

Dequeued: A

Dequeued: B

Current queue: Queue(['C'])

Queue size: 1

Ln 129, Col 1 Spaces: 4 UTF-8 CRLF Python 3.13.1 Python 3.13 (64-bit) Go Live Windsurf Prettier

• Implemented a Queue class using `collections.deque` for efficient operations.

• Methods:

`enqueue(item)`: adds an element to the rear of the queue (FIFO).

dequeue(): removes and returns the front element; raises IndexError if empty.

front(): returns (without removing) the front element; raises IndexError if empty.

size(): returns the current number of elements.

is_empty(): helper to check if the queue has no elements.

Task Description -3 – (Singly Linked List Construction)

- Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Expected Output:

- Correctly functioning linked list with node creation, insertion logic, and display functionality.

```
AI-ASSISTED CODING > task3.py > ...
"""Simple implementation of a Singly Linked List in Python.

This file defines:
- Node class: Represents a single node in the list
- LinkedList class: Manages the list of nodes

Features implemented:
- Insertion at the end of the list
- Traversal / display of all elements

It also includes example usage at the bottom.
"""

class Node:
    """A single node of a singly linked list.

    Each node stores:
    - data: the value we want to keep
    - next: a Reference (link) to the next node in the list, or None

    """
    def __init__(self, data):
        # Store the passed data value in this node
        self.data = data
        # next will point to the next node in the list.
        # For a new node that is not yet connected, it is None.
        self.next = None

class LinkedList:
    """Singly Linked List Implementation.

    Internally, the list is represented by a chain of Node objects.
    The LinkedList keeps a reference to the "first" node, called "head".
    Each node points to the next one, forming a chain.

    head -> [data | next] -> [data | next] -> ... -> None

    """
    def __init__(self):
        # head points to the first node in the list.
        # For an empty list, head is None.
        self.head = None

    def insert_at_end(self, data):
        """Insert a new node with the given data at the end of the list.

        Steps:
        1. Create a new node.
        2. If the list is empty (head is None), make this node the head.
        3. Otherwise, traverse to the last node and link its 'next' to the new node.

        """
        new_node = Node(data)

        # Case 1: The list is empty; the new node becomes the first node (head).
        if self.head is None:
            self.head = new_node
            return

        # Case 2: The list is not empty; find the last node.
        current = self.head
```

(c) Microsoft Corporation. All rights reserved.

```
C:\Users\akhil\OneDrive\Documents\Devops>python -u "C:\Users\akhil\OneDrive\Documents\Devops\AI-ASSISTED_CODING\task3.py"
Inserting elements at the end: 10, 20, 30

Linked list (display):
10 -> 20 -> 30 -> None

Linked list as Python list (traverse):
[10, 20, 30]
```

C:\Users\akhil\OneDrive\Documents\Devops>

- Added Node and LinkedList classes.
- Node holds data and a next reference that points to the next node (or None for the last node).
- LinkedList:

- Maintains head (first node).
- `insert_at_end(data)`: creates a new node and links it at the end by walking from head to the last node and updating its next.
- `traverse()`: walks from head via next, collects data values into a Python list, and returns it.
- `display()`: prints the list in the form `10 -> 20 -> 30 -> None`.

Task Description -4 – (Binary Search Tree Operations)

• **Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.**

Expected Output:

• **BST program with correct node insertion and in-order traversal output.**

```

1  """Binary Search Tree (BST) implementation in Python.
2
3  This file defines:
4  - Node class: represents a single node in the BST
5  - BinarySearchTree class: supports insertion and in-order traversal
6
7  Features implemented:
8  - Insert a value into the BST
9  - In-order traversal (Left, Root, Right)
10
11 Example usage is provided at the bottom.
12 """
13
14
15 class Node:
16     """A single node in a Binary Search Tree.
17
18     Each node stores:
19     - data: the value stored at this node
20     - left: reference to the left child (values < data)
21     - right: reference to the right child (values > data)
22     """
23
24     def __init__(self, data):
25         self.data = data
26         self.left = None
27         self.right = None
28
29
30 class BinarySearchTree:
31     """Binary Search Tree (BST) with recursive insertion and in-order traversal."""
32
33     def __init__(self):
34         # Root of the BST. Initially, the tree is empty so root is None.
35         self.root = None
36
37     def insert(self, value):
38         """Insert a value into the BST.

```

Terminal Output:

```

(c) Microsoft Corporation. All rights reserved.

C:\Users\akhi1\OneDrive\Documents\Devops>python -u "c:\Users\akhi1\OneDrive\Documents\Devops\AI-ASSISTED_CODING\task4.py"
Inserting values into BST: [50, 30, 70, 20, 40, 60, 80]

In-order traversal result:
[20, 30, 40, 50, 60, 70, 80]

Expected sorted order:
[20, 30, 40, 50, 60, 70, 80]

C:\Users\akhi1\OneDrive\Documents\Devops>

```

Chat Window:

DESIGNING A STACK DATA STRUCTURE IN PYTHON

Act as a data structures professor. Help me implement a Binary Search Tree in Python. Requirements:

- Create a Node class
- Implement BST insertion method
- Implement In-order traversal
- Add proper comments explaining recursion
- Provide example with sample values
- Show expected traversal output
- Explain why in-order traversal gives sorted output

Implementing BST in Python

I'll open task4.py (currently empty) and then add a Binary Search Tree implementation with insertion, in-order traversal, comments, and example usage.

Reviewed task4.py

Now I'll write the BST Node and tree classes, plus example and explanations, into task4.py.

Updated task4.py

Here's what I've implemented for your Binary Search Tree:

- Added `Node` and `BinarySearchTree` classes in `AI-ASSISTED_CODING/task4.py`.
- `Node` holds `data`, `left`, and `right` references.
- `BinarySearchTree`...

Added Node and BinarySearchTree classes in AI-ASSISTED_CODING/task4.py.

Node holds data, left, and right references.

BinarySearchTree:

`insert(value)`: public insert method using `_insert_recursive(node, value)`:

Base case: if node is None, create and return a new Node.

Recursive case: go left if value < node.data, right if value > node.data, then return node to maintain links.

inorder_traversal(): returns a list of values; uses _inorder_recursive(node, result):

Base case: node is None → return.

Recursive case: traverse left, visit node (append data), traverse right.

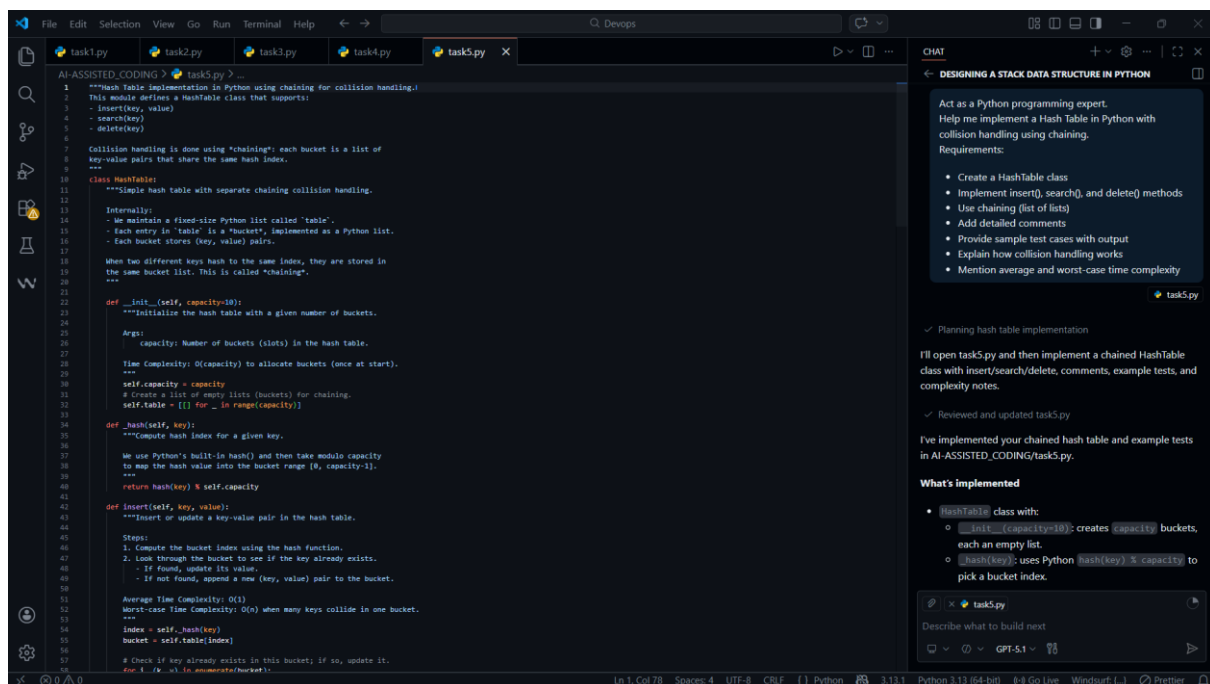
Comments in both recursive helpers explain base/recursive cases clearly.

Task Description -5 – (Hash Table Implementation)

• Task: Create a hash table using AI with collision handling

Expected Output:

• Hash table supporting insert, search, and delete using chaining or open



The screenshot shows a code editor with a file named `task5.py` open. The code implements a `HashTable` class using chaining for collision handling. The class has methods for `insert`, `search`, and `delete`. Comments explain the internal structure, including a 'table' list of buckets and the 'chaining' process. The AI assistant's chat window on the right provides guidance, requirements, and a summary of the implemented code.

```
1 """Hash Table Implementation in Python using chaining for collision handling.
2 This module defines a HashTable class that supports:
3 - insert(key, value)
4 - search(key)
5 - delete(key)
6
7 Collision handling is done using 'chaining': each bucket is a list of
8 key-value pairs that share the same hash index.
9 """
10 class HashTable:
11     """Simple hash table with separate chaining collision handling.
12
13     Internally:
14     - We maintain a fixed-size Python list called 'table'.
15     - Each entry in 'table' is a 'bucket', implemented as a Python list.
16     - Each bucket stores (key, value) pairs.
17
18     When two different keys hash to the same index, they are stored in
19     the same bucket list. This is called 'chaining'.
20     """
21     def __init__(self, capacity=10):
22         """Initialize the hash table with a given number of buckets.
23
24         Args:
25             capacity: Number of buckets (slots) in the hash table.
26
27         Time Complexity: O(capacity) to allocate buckets (once at start).
28         """
29         self.capacity = capacity
30         # Create a list of empty lists (buckets) for chaining.
31         self.table = [[] for _ in range(capacity)]
32
33     def _hash(self, key):
34         """Compute hash index for a given key.
35
36         We use Python's built-in hash() and then take modulo capacity
37         to map the hash value into the bucket range [0, capacity-1].
38         """
39         return hash(key) % self.capacity
40
41     def insert(self, key, value):
42         """Insert or update a key-value pair in the hash table.
43
44         Steps:
45         1. Compute the bucket index using the hash function.
46         2. Look through the bucket to see if the key already exists.
47            - If found, update its value.
48            - If not found, append a new (key, value) pair to the bucket.
49
50         Average Time Complexity: O(1)
51         Worst-case Time Complexity: O(n) when many keys collide in one bucket.
52         """
53         index = self._hash(key)
54         bucket = self.table[index]
55
56         # Check if key already exists in this bucket; if so, update it.
57         for i, (k, v) in enumerate(bucket):
58             if k == key:
59                 bucket[i] = (key, value)
60                 return
61         # Key not found, append new pair to the bucket.
62         bucket.append((key, value))
```

AI-ASSISTED CODING task5.py

Act as a Python programming expert. Help me implement a Hash Table in Python with collision handling using chaining. Requirements:

- Create a HashTable class
- Implement insert(), search(), and delete() methods
- Use chaining (list of lists)
- Add detailed comments
- Provide sample test cases with output
- Explain how collision handling works
- Mention average and worst-case time complexity

✓ Planning hash table implementation

I'll open task5.py and then implement a chained HashTable class with insert/search/delete, comments, example tests, and complexity notes.

✓ Reviewed and updated task5.py

I've implemented your chained hash table and example tests in AI-ASSISTED_CODING/task5.py.

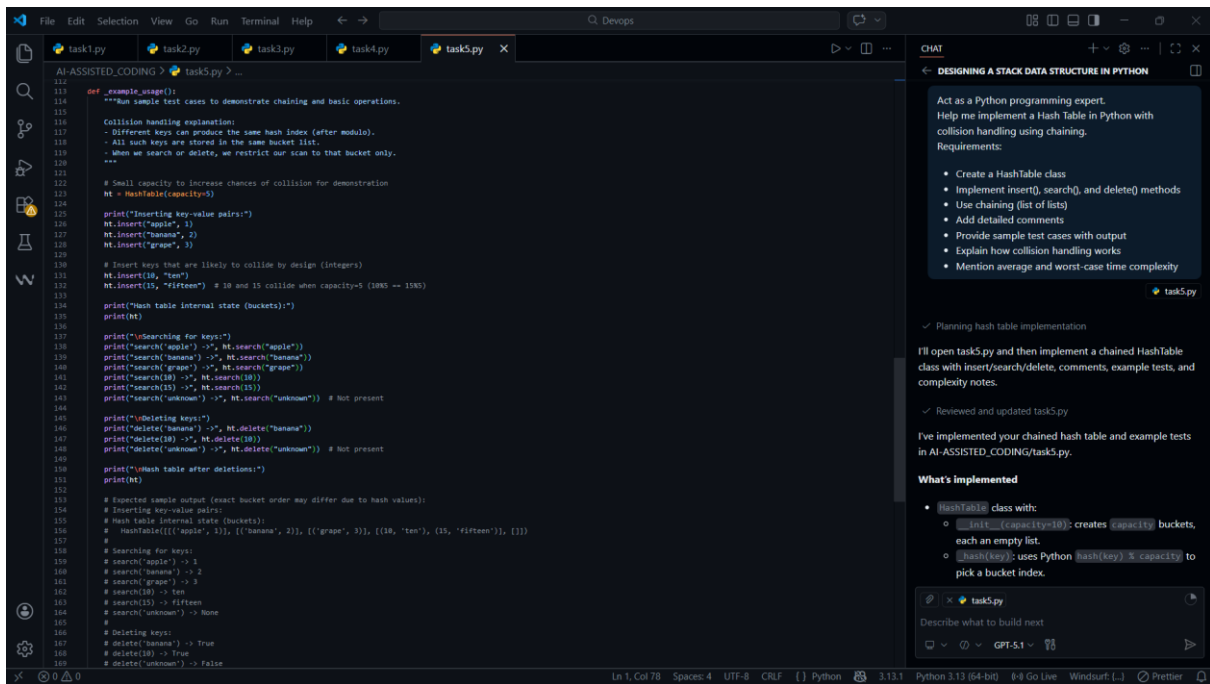
What's implemented

- `HashTable` class with:
 - `__init__(capacity=10)`: creates `capacity` buckets, each an empty list.
 - `_hash(key)`: uses Python `hash(key) % capacity` to pick a bucket index.

task5.py

Describe what to build next

Ln 1, Col 78 Spaces: 4 UTF-8 CRLF Python 3.13.1 Python 3.13 (64-bit) 64 Go Live Windsurf... Premier



Inserting values into BST: [50, 30, 70, 20, 40, 60, 80]

In-order traversal result:
[20, 30, 40, 50, 60, 70, 80]

Expected sorted order:
[20, 30, 40, 50, 60, 70, 80]

In-order traversal result:
[20, 30, 40, 50, 60, 70, 80]

Expected sorted order:
[20, 30, 40, 50, 60, 70, 80]
Expected sorted order:
[20, 30, 40, 50, 60, 70, 80]

C:\Users\akhil\OneDrive\Documents\Devops>python -u "c:\Users\akhil\OneDrive\Documents\Devops\AI-ASSISTED_CODING\task5.py"

Inserting key-value pairs:

Hash table internal state (buckets):

HashTable([(['apple', 1]), (10, 'ten'), (15, 'fifteen')], [], [], [['grape', 3]], [['banana', 2]])

Searching for keys:

search('apple') -> 1
search('banana') -> 2
search('grape') -> 3
search(10) -> ten
search(15) -> fifteen
search('unknown') -> None

Deleting keys:

delete('banana') -> True
delete(10) -> True
delete('unknown') -> False

- HashTable class with:
 - `__init__(capacity=10)`: creates capacity buckets, each an empty list.
 - `_hash(key)`: uses Python `hash(key) % capacity` to pick a bucket index.
 - `insert(key, value)`: updates existing key or appends (key, value) into the bucket.
 - `search(key)`: scans the bucket for key, returns the value or None.

- `delete(key)`: removes (key, value) from the bucket, returns True/False.
- Chaining:
 - `self.table` is a list of lists (buckets).
 - Each bucket stores multiple (key, value) pairs that share the same index → this is collision handling by chaining.